GarantiyAid mobile

Update and completion guide

Prepared for Ronan | Mobile capstone assignment | October 19, 2026

This guide records what is implemented, what changed most recently, and how to run and test the app. It covers the beneficiary mobile app and its supporting local API. The staff web application is a separate team responsibility.

What is working now

The main account journey uses PostgreSQL: create an account, verify a development OTP, sign in, restore the session, view saved details, check enrollment and schedules, present a server-issued QR pass when eligible, and read claim and wallet records.

Current result	Evidence
10 mobile regression tests pass	Session races, account switching, token storage, and Wallet states
4 backend tests pass	Phone normalization, secret hashes, registration input, and date validation
Both TypeScript checks pass	Mobile and backend source compile without type errors
Web and Android exports pass	Browser JavaScript and Android Hermes bundles build successfully

Most recent update

1. Replaced simulated claim and wallet records with authenticated API data.
2. Added saved transaction history and receipt details.
3. Removed the beneficiary action that fabricated a successful claim.
4. Prevented old requests from restoring data after logout or overwriting another account.
5. Corrected Wallet loading/error/empty states and the registration storage notice.

Status meaning: Completed means implemented and checked in the local project. Preview means a screen or interaction exists but its real service is not connected. The app is not being described as production-ready.

1. Completed: mobile account journey

These features form the working foundation of the beneficiary app. Saved account records come from the backend rather than hard-coded profile examples.

Completed task	What the user can do
Mobile interface and navigation	Use the Figma-based blue/green interface and Home, Wallet, Help, and Profile tabs. The browser presents a phone-sized app layout.
Welcome and account choice	Open the app at Welcome and choose account creation or sign-in. Opening the app does not automatically submit a registration.
Four-step account creation	Enter name, birth date, sex, address, barangay/service area, city, and mobile number, then verify the generated OTP.
Form validation	Receive field errors for invalid identity, address, phone, or date inputs. Select active barangays loaded from the API.
Development OTP login	Request a generated code and sign in with the registered phone number. Expiry, retry limits, and resend cooldown are implemented. SMS delivery is still unconnected.
Persistent session	Reopen the app and restore a valid saved login. Native tokens use Expo SecureStore; the browser preview uses local storage.
Logout and protected tabs	Revoke the server session, clear local account state, and return to the signed-out experience. Tab routes require a valid local session.
Saved beneficiary details	View the server's name, phone, address, barangay/service area, date of birth, verification flag, and account status. Saving profile edits is not connected yet.
English/Bisaya support	Switch the app's labels, instructions, validation messages, and supported assistant responses between English and Bisaya.

Account data notice

The registration notice now explains that details are sent to GarantiyAid and saved when the account is created. The old statement that registration details stay only in the app session has been removed.

2. Completed: schedules, claims and wallet

The mobile app reads the signed-in beneficiary's records. Authorized staff remain responsible for assigning schedules and recording actual claim outcomes.

Completed task	Current behavior
PostgreSQL and API foundation	Express and Prisma store beneficiary accounts, authentication sessions, enrollments, schedules, QR tokens, claims, wallet accounts, and transactions. Migrations and a local seed are included.
Program enrollment status	Show the enrolled program, enrollment date, grant amount, and recorded enrollment status. This is a display of saved enrollment, not a mobile approval process.
Home and schedule details	Show the next eligible assigned distribution, Manila-time slot, location, queue number, and schedule status. Handle missing or failed records.
Secure QR claim pass	Request an opaque pass from the server. The database stores its hash; replacement issuance revokes the previous active pass. Expiry follows the assigned slot.
Saved claim status	Read the current scheduled claim and recorded claim history through the protected wallet API. Help can answer from saved claim data.
Wallet and transaction history	Display the saved ledger balance and recorded assistance entries, including program, amount, timestamp, status, and reference.
Receipts	Open a transaction in Wallet or select a history entry to view its recorded receipt. Receipts use transaction amounts and references.
Recovery states	Use loading indicators, empty states, error messages, retry controls, and record refresh actions. The app cannot approve its own claims.

Local test records

The seed contains a fictional test account, `0917 555 0147`, with approved 4Ps enrollment, a completed August 28, 2026 claim, and a **PHP 1,500.00** ledger entry. The receipt reference is `GAI-2026-0828-032`. These are local sample records, not evidence of a real government payment.

The fixed October 2, 2026 schedule has expired as of this guide. No upcoming schedule or available QR pass is therefore expected for that seed. A future assigned schedule is needed to demonstrate fresh QR issuance again.

Wallet boundary: the app records assistance. It does not hold money or initiate bank or e-wallet transfers.

3. Latest fixes and verification

The latest reliability work protects the account journey when requests finish late or a user switches accounts.

Fix completed	Result
Ignore obsolete session responses	Wallet and schedule responses from before logout cannot restore cleared account data.
Protect a replacement login	Late success and HTTP 401 responses cannot overwrite or sign out a new account. Logout also cancels unfinished session restoration.
Preserve token-write order	Native saves/clears run in order; browser reads wait for writes. A failed save does not block logout.
Correct Wallet states	"No wallet history yet" appears only after a successful load with no entries.
Correct registration information	English and Bisaya notices explain that account details are saved to the server.

Checks completed

Check	Recorded result
`mobile: npm test`	PASS - 10 regression tests with controlled requests, rendering, and storage
`backend: npm test`	PASS - 4 input/security tests
`npm run typecheck` in both projects	PASS
Web and Android exports	PASS - browser JavaScript and Android Hermes bundles
Authenticated wallet API	PHP 1,500 balance, one completed claim, one receipt
Access without login / after logout	HTTP 401 in both cases
Expired schedule / QR	No upcoming schedule; QR request returns HTTP 404

What these checks do not establish

An Android bundle export is not an installed APK or a physical-device test. A complete visual browser review of the latest fixes is still pending. Camera permissions, system Back, keyboard resize, safe areas, and large text still need an Android walkthrough. Staff claim approval, SMS, push delivery, and real biometric matching are not connected.

4. Step by step: start the backend

Use PowerShell in VS Code. The project folder on this computer is `C:\Users\Ronan\Documents\Garantiy-Aid`. Keep the backend running while testing the mobile app.

Step 1. Open the project and install backend packages

Open the project folder in VS Code, open a terminal, then run:

```
cd C:\Users\Ronan\Documents\Garantiy-Aid\backend
npm install
```

Step 2. Keep or create the backend environment file

Keep an existing configured `backend/.env`. On a first setup only, copy `.env.example` to `.env`. It contains the database URL, port, allowed browser origins, and `AUTH_PEPPER`. Never upload the private `.env` file.

Step 3. Start the local PostgreSQL service

```
npm run db:local
```

Use the **direct** `postgres://` **connection string** printed by this command for `DATABASE_URL` in `backend/.env`. The current local server normally uses port 51214. Copy the URL printed on your machine rather than assuming a port. Do not use the `prisma+postgres://` URL with this project's PostgreSQL driver.

Step 4. Set the authentication secret on a first setup

Set `AUTH_PEPPER` to a random value of at least 32 characters. To generate one locally:

```
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

Paste the result into `backend/.env`. Keep an existing configured value when returning to the project.

Step 5. Generate the client, apply migrations, and start the API

For local test data, run:

```
npm run db:generate
npm run db:deploy
npm run db:seed
npm run dev
```

The seed sets up the fictional test account and its fixed example records. Open the health endpoint; expect `status: ok` and `database: connected`. Leave the API terminal open.

5. Step by step: run and test the app

Step 6. Start the browser preview in a second terminal

Keep or create `mobile/.env` from `.env.example`. For this browser, set `EXPO_PUBLIC_API_URL=http://127.0.0.1:3000`, then run:

```
cd C:\Users\Ronan\Documents\Garantiy-Aid\mobile
npm install
npm run web -- --port 8765
```

Open the mobile browser preview. If the API address changes, restart Expo so it reloads the environment file.

Step 7. Follow this testing checklist

Action	Expected result
Open while signed out	Welcome offers account creation and sign-in; it does not automatically register an account.
Create a new test account	Complete the identity/address/phone steps and enter the generated development OTP. The account details are saved to the local database.
Sign in to the seeded account	Use `0917 555 0147`, request the code, then enter the development OTP displayed by the app or backend terminal. There is no permanent shared OTP.
Check Profile and enrollment	Seeded details and approved 4Ps enrollment appear. A newly created account does not inherit the seed's enrollment or wallet records.
Check Home, Schedule, and QR	With the expired October 2 seed, no upcoming schedule or QR pass is available. A future staff-assigned schedule is required for an active pass.
Check Wallet and receipts	The seeded account shows PHP 1,500.00 and reference `GAI-2026-0828-032`. Open the receipt and confirm its program, timestamp, status, and amount.
Check a new account's Wallet	A new account without recorded assistance shows zero recorded balance and an empty history after a successful load.
Ask Help about schedule/claim status	Answers use saved account records. Other assistant topics remain fixed preview responses.
Refresh, log out, then sign in again	Valid records reload. Logout clears local data and revokes the server session. Late results must not restore old records.

Step 8. Repeat the automated checks when making changes

Run `npm test` and `npm run typecheck` inside both projects. In `mobile`, also run `npm run export:web` and `npm run export:android` before sharing a build. Browser testing is available now; native-device verification remains a separate step.

6. Remaining work and next tasks

These items are not marked as completed services, even where a prototype screen already exists.

Area	Present state	Work remaining
In-app notifications	Sample list, details, and session-only read state	Load beneficiary notifications from the API; save read/unread status; refresh the badge and details. This is the recommended next mobile task.
SMS and push delivery	Development OTP codes; no delivery provider	Connect an SMS provider and optional device push registration/delivery through the backend.
Staff QR scanning and claim approval	Mobile pass issuance and read-only results are complete	The web/backend team must validate the pass, block duplicates, and atomically record the claim, transaction, wallet, schedule, and used pass.
Biometrics and consent	Optional consent/camera/demo enrollment UI	Store versioned consent, handle permitted uploads, and connect matching/liveness. Current face photos are not uploaded.
Enrollment documents	Local file selection preview	Upload permitted files, show server validation, and connect staff review and enrollment actions.
Profile changes and recovery	Saved detail views; password/recovery previews	Agree allowed profile updates and connect recovery/authentication behavior. Phone OTP is the working login method today.
Assistant and staff support	Saved schedule/claim answers; other static responses	Connect approved live assistance/escalation services if required by the team. No support message is currently sent.
Android release validation	Android source bundle compiles	Test on Expo Go or a supported native build, then verify permissions, camera, Back, keyboard, text scaling, and safe areas.

Recommended order

1. Save this milestone in GitHub with the completed claim/wallet work and reliability fixes.
2. Implement real in-app notifications and persisted read/unread status.
3. Agree shared claim/QR/document API behavior with the web/backend teammate.
4. Complete the remaining service integrations and run the Android walkthrough.

Project references

Repository: [RonanStack24 / Garantiy-Aid]

Implementation notes: `mobile/README.md`, `backend/README.md`, `mobile/docs/backend-contract.md`, and `mobile/docs/verification.md`. The editable source of this report is `mobile/docs/app-update-and-testing-guide.md`.